

GROUP NO 24

Pankaj Singh	20251651067
Paras Pandey	20251651068
Pranav Bhavsar	20251651069
Jayesh Badole	20251651049
Shivam Phogat	20251651082

CampusEats

Services, Contracts & Schema — Design Benchmark

This document is the design benchmark for CampusEats: the capabilities the system performs, the services those capabilities are grouped into, the contract each service exposes to the others, the full specification of the central placeOrder operation, and the service-property validation.

The companion files `services.drawio` / `services.png`, `schema.drawio` / `schema.png`, and `schema.sql` are the diagram sources and outputs referenced throughout.

1. Capabilities

Every distinct job the CampusEats system does, read off the Assignment 1 brief (a campus food-ordering platform connecting students/staff, campus food vendors, and delivery

riders). Each capability is a verb — something the system *does* — before any service boundary is drawn.

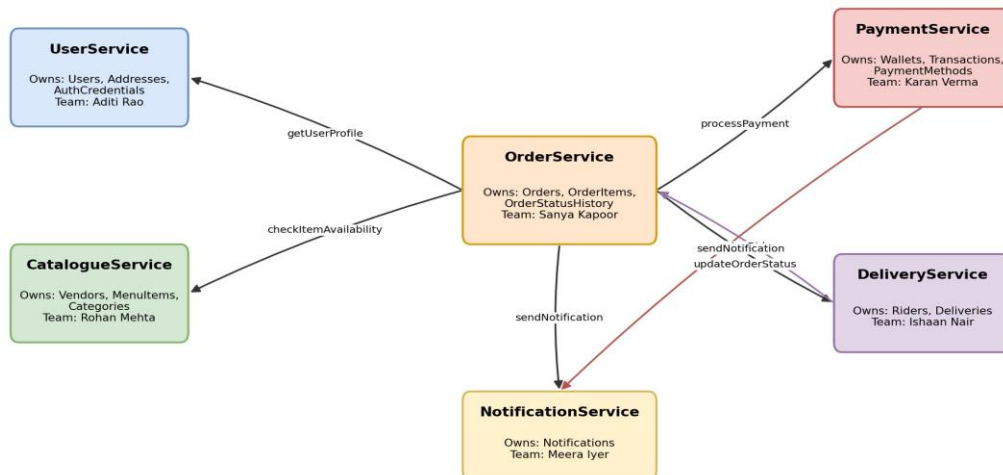
Capability (verb)	What it covers
Register & authenticate users	Create an account (student/staff/vendor-admin), log in, manage saved delivery addresses.
Browse vendors & menus	List campus food vendors, view a vendor's menu items, prices and availability.
Manage menu & availability	Vendor-side: add/update menu items, categories, prices, and mark items in/out of stock.
Place an order	Validate cart items, compute total, and create an order against a chosen vendor and address.
Track order status	Let a user or vendor query the current status and status history of an order.
Process payment	Charge a wallet/payment method for an order and record the transaction; support refunds.
Assign & track delivery	Assign an available rider to a confirmed order and track the delivery in real time.
Cancel an order	Allow cancellation while still eligible, releasing payment and delivery holds.
Send notifications	Notify users of order confirmation, payment receipts, and delivery milestones.

That is 9 capabilities — comfortably above the required minimum of six — used below as the basis for grouping by data ownership into services.

2. Service Design

Capabilities were grouped by the data they read/write, giving six services with disjoint data ownership (no two services share a table). See **services.drawio** (editable) and **services.png / services.pdf** (exported) for the full boxes-and-arrows diagram, reproduced below at reduced size.

CampusEats — Service Design (Boxes = Services, Arrows = Cross-Service Calls)



Rule: no two services share data. Each box lists the data (tables) it, and only it, owns.

Service	Team member	Data owned
UserService	Paras Pandey	Users, Addresses, AuthCredentials
CatalogueService	Jayesh Badole	Vendors, MenuItems, Categories
OrderService	Pankaj Singh	Orders, OrderItems, OrderStatusHistory
PaymentService	Pranav Bhavsar	Wallets, Transactions, PaymentMethods
DeliveryService	Shivam Phogat	Riders, Deliveries

NotificationService	Pankaj Singh	Notifications
---------------------	--------------	---------------

Cross-service calls (also shown as labelled arrows in services.drawio):

Call	Operation	Purpose
OrderService → UserService	getUserProfile	Fetch delivery address & profile for order placement
OrderService → CatalogueService	checkItemAvailability	Validate items and lock in price before creating the order
OrderService → PaymentService	processPayment	Charge the user for the order total
OrderService → DeliveryService	assignRider	Request a rider once payment succeeds
DeliveryService → OrderService	updateOrderStatus	Push delivery milestones back onto the order
OrderService → NotificationService	sendNotification	Order confirmation / cancellation alerts

PaymentService → NotificationService	sendNotification	Payment receipt / refund alerts
--	------------------	---------------------------------------

3. Service Contracts

Each table below is the contract a service exposes to callers: operation, input, output and error cases. Contracts describe the operation only — no table names, id formats, or SQL leak through.

UserService

Owns: Users, Addresses, AuthCredentials

Operation	Input	Output	Errors
registerUser	name, email, password, role	userId, profile summary	EMAIL_ALREADY_EXISTS, INVALID_INPUT
authenticateUser	email, password	authToken, userId	INVALID_CREDENTIALS, ACCOUNT_LOCKED
getUserProfile	userId	name, defaultAddress, contactInfo	USER_NOT_FOUND

CatalogueService (main)

Owns: Vendors, MenuItems, Categories

Operation	Input	Output	Errors
listVendors	campusZone (optional), filter	list of {vendorId , name, rating, isOpen}	INVALID_FILTER

getMenu	vendorId	list of {itemId, name, price, category, inStock}	VENDOR_NOT_FOUND
checkItemAvailability	vendorId, itemId, quantity	available (bool), unitPrice	VENDOR_NOT_FOUND, ITEM_NOT_FOUND
updateItemAvailability	vendorId, itemId, available Qty	confirmation	VENDOR_NOT_FOUND, ITEM_NOT_FOUND, UNAUTHORIZED

OrderService (main, central)

Owns: Orders, OrderItems, OrderStatusHistory

Operation	Input	Output	Errors
placeOrder	userId, vendorId, items[], deliveryAddressId, paymentMethod	orderId, status, estimatedDeliveryTime, totalAmount	see full spec § 4
getOrderStatus	orderId	status, statusHistory, estimatedDeliveryTime	ORDER_NOT_FOUND

cancelOrder	orderId, requestedBy	confirmation, refundTriggered (bool)	ORDER_NOT_FOUND, CANNOT_CANCEL
updateOrderStatus	orderId, newStatus	confirmation	ORDER_NOT_FOUND, INVALID_TRANSITION

PaymentService

Owns: Wallets, Transactions, PaymentMethods

Operation	Input	Output	Errors
processPayment	orderId, userId, amount, method	transactionId, status	INSUFFICIENT_BALANCE, PAYMENT_DECLINED, INVALID_METHOD
refundPayment	transactionId, amount	refundId, status	TRANSACTION_NOT_FOUND, REFUND_NOT_ALLOWED
getWalletBalance	userId	balance	USER_WALLET_NOT_FOUND

DeliveryService

Owns: Riders, Deliveries

Operation	Input	Output	Errors
assignRider	orderId, pickupVendorId,	deliveryId, riderId, eta	NO_RIDER_AVAILABLE

	dropAddressId		E, ORDER_ALREADY_ASSIGNED
getDeliveryStatus	deliveryId	status, riderLocation, eta	DELIVERY_NOT_FOUND
updateDeliveryStatus	deliveryId, status	confirmation	DELIVERY_NOT_FOUND, INVALID_STATUSES

NotificationService

Owms: Notifications

Operation	Input	Output	Errors
sendNotification	userId, type, payload	notificationId, status	USER_NOT_FOUND, TEMPLATE_NOT_FOUND

4. Full Specification — placeOrder

placeOrder is CampusEats' central operation: it is the one call that touches every other service, either directly or via a follow-on event.

Inputs

Field	Description
userId	The ordering user, must already be registered

vendorId	The vendor the order is placed against
items[]	List of {itemId, quantity} pairs — the cart
deliveryAddressId	One of the user's saved addresses
paymentMethod	Wallet or a registered payment method to charge

Outputs

Field	Description
orderId	Newly created order's identifier
status	Initial status returned, e.g. CONFIRMED
totalAmount	Computed total (items + any fees) actually charged
estimatedDeliveryTime	ETA returned by the assigned rider

Error cases

Error	Cause
USER_NOT_FOUND	userId does not resolve to a real, active account
VENDOR_NOT_FOUND / VENDOR_CLOSED	Vendor does not exist or is not currently accepting orders
ITEM_UNAVAILABLE	One or more items are out of stock or priced differently than the cart assumed
INVALID_ADDRESS	deliveryAddressId does not belong to userId

INSUFFICIENT_BALANCE / PAYMENT_DECLINED	Payment could not be captured
NO_RIDER_AVAILABLE	Payment succeeded but no rider could be assigned within the timeout

Internal details placeOrder must hide from callers

- Calling UserService.getUserProfile to validate the user and resolve the delivery address, rather than the caller supplying address text directly.
- Calling CatalogueService.checkItemAvailability per line item to re-price and re-validate stock at order time (cart prices are never trusted as final).
- Computing the order total, persisting the order and its line items with status PENDING before payment is attempted.
- Calling PaymentService.processPayment synchronously; on failure, marking the order CANCELLED and returning the payment error — never leaving a PENDING order with no payment outcome.
- Calling DeliveryService.assignRider only after payment succeeds, and handling the case where no rider is available by refunding the payment (via PaymentService.refundPayment) and cancelling the order.
 - Appending every state change to the order's status history and firing an asynchronous sendNotification call — a failure to notify must never fail the order itself.
 - Any concurrency control needed on menu-item stock counts (e.g. two students ordering the last portion) — CatalogueService owns and resolves this, OrderService only sees the yes/no answer.

- Retry, timeout and idempotency handling for the downstream Payment and Delivery calls, so a network blip cannot double-charge a user or create two riders for the same order.

5. Service Property Validation

Each service checked against the five properties of a Service:
reachable, self-contained, has a contract, independent,
loosely coupled.

Service	Reac ha ble	Self - con tai ned	Has contr act	Indep end ent	Loosely coupled	Fix if failing
UserServi ce	Yes	Yes	Yes	Yes	Yes	—
Catalogue Servi ce	Yes	Yes	Yes	Yes	Yes	—
OrderServ ice	Yes	Yes	Yes	Yes	Partial — synchron ously calls Payment and Delivery in placeOrd er	Fire- and- forget the Delivery- assignm ent and Notificati on steps as events after payment succeed s, so OrderSe rvice blocks on

						Payment only.
PaymentService	Yes	Yes	Yes	Yes	Yes	—
DeliveryService	Yes	Yes	Yes	Yes	Yes	—
NotificationService	Yes	Yes	Yes	Yes	Yes	—

How each property was checked: *Reachable* — every service exposes its operations over a network endpoint, none require direct DB access from another service. *Self-contained* — each service holds all logic and data needed to execute its own operations without reaching into another service's tables.

Has a contract — § 3 lists a stable

operation/input/output/error contract for every service.

Independent — each service can be deployed and scaled on its own schema. *Loosely coupled* — checked by counting synchronous calls a service makes to complete one operation; OrderService is the one service that fails this because placeOrder chains four synchronous calls end-to-end.